SOFTWARE REQUIREMENTS SPECIFICATION

Finwave - FinTech Financial Inclusion Platform

for Migrant Workers

Project Title	FinTech Financial Inclusion Platform for Migrant Workers
Document Type	Software Requirements Specification (SRS)
Team Number	Team 20
Version	1.0.0
Date	22 May 2026
Status	Final – Hackathon Submission
Classification	Public

Prepared for:
Hackathon Judging Committee

Table of Contents

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the complete functional and non-functional requirements for the FinTech Financial Inclusion Platform for Migrant Workers — a mobile-first digital financial services application developed as a Hackathon prototype by Team 20.

The purpose of this document is to provide a comprehensive, unambiguous, and complete specification of the system's intended behavior, capabilities, constraints, and interfaces. It serves as a contract between the development team and all project stakeholders, including judges, sponsors, and potential investors.

1.2 Scope

The platform, referred to henceforth as 'FinWave', is a cross-platform, cloud-hosted fintech application providing zero-fee international remittances, secure digital wallet management, AI-powered financial insights, micro-investment tools, and multilingual support to migrant workers and financially underserved populations globally.

The scope of this prototype includes:

- A mobile-first React Progressive Web Application (PWA)
- A RESTful Node.js/Express.js backend API
- A MongoDB Atlas cloud database
- Firebase Authentication with OTP/SMS verification
- AI-powered insights engine via OpenAI/Gemini API integration
- Simulated blockchain transaction verification layer
- Multi-language support using react-i18next
- Cloud hosting on (frontend) and Render/Railway (backend)

1.3 Intended Audience

Audience	Role	Relevance
Hackathon Judges	Evaluators	Assess technical depth, innovation, and completeness
Development Team	Builders	Reference for implementation and testing
Potential Investors	Stakeholders	Understand business value and technical feasibility
Academic Faculty	Reviewers	College project assessment and grading
End Users	Migrant Workers	Understand platform capabilities and benefits

1.4 Problem Statement

Migrant workers represent one of the most financially vulnerable demographics globally. According to the World Bank, migrants remit over $700 billion annually, yet pay an average transfer fee of 6.2% — a disproportionate tax on the world's lowest-income earners. Key challenges include:

- High remittance costs: Traditional transfer services charge 5–10% per transaction
- Lack of banking access: Many migrants remain unbanked or underbanked
- Financial illiteracy: Limited access to financial education and planning tools
- Language barriers: Financial platforms are rarely available in regional migrant languages
- No investment access: Micro-savings and investment products are inaccessible to low-income workers

1.5 Objectives

FinWave aims to achieve the following primary objectives:

1. Eliminate remittance fees by providing zero-cost money transfer infrastructure
2. Deliver personalized AI financial coaching accessible to low-literacy users
3. Provide micro-investment opportunities starting from as little as $1
4. Bridge language barriers through comprehensive multilingual support
5. Build trust through simulated blockchain transaction transparency

1.6 Definitions, Acronyms, and Abbreviations

Acronym / Term	Full Form / Definition
SRS	Software Requirements Specification
API	Application Programming Interface
OTP	One-Time Password
KYC	Know Your Customer
JWT	JSON Web Token
AI	Artificial Intelligence
UI/UX	User Interface / User Experience
REST	Representational State Transfer
MFA	Multi-Factor Authentication
AES	Advanced Encryption Standard
TLS	Transport Layer Security
GDPR	General Data Protection Regulation
PWA	Progressive Web Application
HTTP	Hypertext Transfer Protocol
CORS	Cross-Origin Resource Sharing

1.7 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- World Bank Remittance Prices Worldwide Report, 2024
- Firebase Authentication Documentation: https://firebase.google.com/docs/auth

- OpenAI API Reference: https://platform.openai.com/docs
- MongoDB Atlas Documentation: https://www.mongodb.com/docs/atlas
- React Documentation: https://react.dev
- Node.js/Express.js Documentation: https://expressjs.com
- react-i18next Documentation: https://react.i18next.com

1.8 Document Overview

This document is organized into 21 sections following the IEEE 830-1998 SRS standard. Sections 1–2 introduce the project and provide a high-level overview. Sections 3–5 detail system features and requirements. Sections 6–13 cover technical architecture, design, and security. Sections 14–16 present behavioral models. Sections 17–21 address future scope, advantages, limitations, and conclusions.

2. Overall Description

2.1 Product Perspective

FinWave is a standalone, greenfield fintech platform designed as an independent system for the hackathon context. In a production deployment, it would interface with real banking rails, international payment gateways (SWIFT/SEPA), and regulatory KYC/AML services. The system operates within a three-tier architecture comprising a client-side PWA, a server-side RESTful API, and a cloud-hosted NoSQL database.

The platform is conceived as a potential competitor and alternative to Western Union, Wise, and Remitly, with a core differentiator of zero-fee transfers enabled through a float-based or cross-border netting revenue model in its production version.

2.2 Product Functions

At a high level, FinWave provides the following major functional areas:

Function Area	Description
User Onboarding	Secure registration, Firebase OTP authentication, and digital identity creation
Digital Wallet	Multi-currency wallet with real-time balance tracking and transaction history
Zero-Fee Remittance	Instant money transfers with blockchain-verified transaction records
AI Financial Insights	Personalized spending analysis, savings recommendations, and investment nudges
Micro-Investments	Low-barrier investment product suggestions and portfolio tracking
Multilingual Interface	Full UI localization supporting 10+ languages via react-i18next
Notifications	Push and in-app alerts for transactions, insights, and account activity
Security & Compliance	Encrypted data, JWT sessions, OTP MFA, and KYC data capture

2.3 User Classes and Characteristics

User Class	Characteristics	Technical Literacy	Primary Use
Migrant Worker (Primary)	Ages 18–55, mobile-first, low-to-medium income, often multilingual	Low–Medium	Send money, check balance, view insights
Recipient (Beneficiary)	Family members in home country, mobile or no-internet access	Very Low	Receive notifications of incoming funds
Power User	Financially literate migrant worker interested in saving and investing	Medium–High	Use AI tools, micro-investments, full dashboard
System Administrator	Platform operator, handles compliance and user management	High	Admin panel, user management, reporting

2.4 Operating Environment

- Client Devices: Android and iOS smartphones, tablets, and desktop browsers

- Minimum Browser Support: Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
- Network Conditions: Designed to function on 3G/4G; graceful degradation on 2G
- Frontend Hosting: Cloudfront CDN with global edge caching
- Backend Hosting: EC2 (Node.js container)
- Database: MongoDB Atlas M0 (free tier) to M10+ for production scaling
- Authentication: Google Firebase Authentication (cloud-hosted, globally distributed)
- AI API: OpenAI GPT-4o-mini or Google Gemini 1.5 Flash (REST API, cloud-hosted)

2.5 Design Constraints

- Hackathon Constraint: Prototype must be built and deployed within 24–48 hours
- Blockchain: Real distributed ledger is not feasible in the timeframe; a simulated ledger with cryptographic hashing is used
- Banking Rails: Real banking integrations require regulatory approval; mock APIs simulate fund transfers
- KYC: Full KYC verification requires third-party integrations; basic identity capture is implemented
- Cost: All infrastructure must use free-tier or minimal-cost cloud services
- Regulatory: Actual money transmission licenses are not in scope for the prototype

2.6 Assumptions and Dependencies

Assumptions:

- Users have access to a smartphone with a modern browser or the FinWave PWA installed
- Users have a valid mobile phone number for OTP verification
- Internet connectivity is available, even if intermittent
- The OpenAI/Gemini API key is available and within rate limits during the hackathon demo

Dependencies:

- Firebase Authentication service availability (99.95% SLA)
- MongoDB Atlas cluster availability (99.995% SLA for M10+)
- OpenAI/Gemini API availability for AI insight generation
- AWS platform availability for hosting
- react-i18next library for internationalization

3. System Features

This section details every major feature of the FinWave platform. Each feature is described with its inputs, outputs, functional behavior, and development priority using a High/Medium/Low classification.

3.1 User Registration

Description	Allows new users to create a FinWave account by providing personal details and verifying their mobile number via OTP.
Inputs	Full name, email address, mobile number, country of origin, preferred language, password
Outputs	Newly created user account, Firebase UID, Digital wallet initialized, Welcome notification triggered
Functional Behavior	User submits registration form → System validates inputs → Firebase sends OTP to mobile → User verifies OTP → Backend creates User document in MongoDB → Wallet document created with zero balance → JWT issued → User redirected to onboarding dashboard
Priority	HIGH

3.2 Firebase OTP Authentication

Description	Provides secure two-factor authentication using Firebase's phone number OTP service, ensuring each login is verified against a registered mobile number.
Inputs	Registered mobile phone number, OTP code (6 digits, time-limited to 5 minutes)
Outputs	Firebase ID token, Backend JWT access token, Refresh token, Session established
Functional Behavior	User enters phone number → Firebase reCAPTCHA verification → OTP dispatched via SMS → User inputs OTP → Firebase verifies OTP → Backend exchanges Firebase token for application JWT → Secure session established
Priority	HIGH

3.3 Digital Wallet

Description	A secure, multi-currency digital wallet associated with each user account. Serves as the central financial hub for all platform transactions.
Inputs	User account ID, currency preference, initial deposit (if applicable)
Outputs	Wallet balance (real-time), currency code, wallet ID, transaction history link
Functional Behavior	Wallet is created automatically on registration. Users can view balance, add funds (simulated), receive transfers, and initiate outgoing transfers. All balance updates are atomic MongoDB transactions to prevent race conditions.
Priority	HIGH

3.4 Zero-Fee Remittance

Description	The platform's flagship feature — allows users to send money internationally with zero transfer fees. Revenue model in production would use float income and FX spread.
Inputs	Sender wallet ID, recipient identifier (phone/email), amount, destination currency, optional note
Outputs	Transaction confirmation, blockchain verification hash, updated sender and recipient balances, receipt PDF (future)
Functional Behavior	Sender initiates transfer → System validates balance sufficiency → FX conversion rate applied (simulated) → Debit sender wallet → Credit recipient wallet → Blockchain simulation module generates transaction hash → Both parties notified → Transaction recorded in history
Priority	HIGH

3.5 Send / Receive Money

Description	Peer-to-peer domestic and cross-border money transfer between registered FinWave users, with support for QR code-based recipient identification.
Inputs	Sender user ID, recipient user ID or QR code, amount, currency, transfer note
Outputs	Success/failure response, transaction ID, updated wallet balances, push notification to both parties
Functional Behavior	User selects 'Send Money' → Enters recipient identifier or scans QR → Inputs amount → Confirms transaction with biometric or PIN (future) → System processes transfer → Both wallets updated → Notifications dispatched
Priority	HIGH

3.6 Transaction History

Description	A chronological, filterable, and searchable log of all wallet transactions associated with a user account.
Inputs	User ID, optional filters (date range, type, amount range, status)
Outputs	Paginated list of transaction objects, total count, export option (CSV/PDF in future scope)
Functional Behavior	User navigates to History tab → System fetches transactions from MongoDB with pagination (20 per page) → Results rendered with color-coded status badges (completed, pending, failed) → User can search or filter → Blockchain hash visible per transaction for verification
Priority	HIGH

3.7 Blockchain Verification Badge

Description	Each transaction is assigned a simulated blockchain hash (SHA-256) stored in a simulated distributed ledger, providing a transparency and trust mechanism for users.

Inputs	Transaction data object (sender, receiver, amount, currency, timestamp)
Outputs	SHA-256 hash string, verification status (Verified / Pending), ledger entry ID
Functional Behavior	On transaction completion, the Blockchain Simulation Module hashes the transaction data using SHA-256 → Hash stored in Blockchain collection → Transaction document linked to hash → Users can view and copy hash → 'Verified' badge displayed on transaction card
Priority	MEDIUM

3.8 AI-Powered Financial Insights

Description	Leverages the OpenAI GPT-4o-mini or Google Gemini API to analyze user spending patterns and generate personalized, actionable financial recommendations.
Inputs	User transaction history (last 90 days), wallet balance, savings goals (if set), preferred language
Outputs	Categorized spending analysis, top 3 savings recommendations, investment nudges, risk assessment, plain-language financial tips
Functional Behavior	User visits Insights tab → System aggregates transaction data → Structured prompt sent to AI API with anonymized financial data → AI returns analysis JSON → Frontend renders insights as cards with charts → Insights cached for 24 hours to minimize API calls
Priority	HIGH

3.9 Savings Recommendations

Description	Analyzes income and expenditure patterns to suggest personalized savings targets and strategies aligned with user goals (e.g., sending more home, building emergency fund).
Inputs	Monthly income estimate, expense categories, remittance frequency, savings goal (optional)
Outputs	Recommended monthly savings amount, percentage of income to save, goal timeline, motivational milestone tracker
Functional Behavior	AI module identifies discretionary spending → Computes potential savings capacity → Generates goal-specific recommendations → Displayed as an interactive savings plan with progress visualization
Priority	MEDIUM

3.10 Expense Tracking

Description	Automatically categorizes outgoing transactions into spending buckets (Food, Remittance, Housing, Transport, Leisure, etc.) and presents a visual spending overview.
Inputs	All outgoing transactions with amounts and timestamps
Outputs	Categorized expense breakdown (pie/bar chart), monthly total per category, comparison with previous month

Functional Behavior	Transactions tagged with categories (AI-assisted or rule-based) → Dashboard renders category pie chart → Monthly and weekly views available → Alerts when spending in a category exceeds user-defined threshold
Priority	MEDIUM

3.11 Micro-Investment Suggestions

Description	Recommends low-risk micro-investment products (simulated ETFs, savings bonds, gold units) to users who have surplus balance, making investing accessible from as little as $1.
Inputs	Wallet balance, risk tolerance (user-set or AI-inferred), investment history
Outputs	List of simulated investment products with expected returns, risk rating, minimum investment, and one-tap invest button (simulated)
Functional Behavior	AI analyzes surplus balance → Recommends 3 simulated products → User reviews product card → Taps 'Invest' → Amount deducted from wallet → Investment ledger entry created → Dashboard shows portfolio value
Priority	MEDIUM

3.12 Multilingual Support

Description	Full UI localization using react-i18next, supporting 10+ languages commonly spoken by migrant worker populations.
Inputs	User's preferred language (set at registration or updated in settings), browser locale detection
Outputs	Fully translated UI in selected language, RTL layout support for Arabic/Urdu, persisted language preference
Functional Behavior	Language detected on first launch → User confirms or changes preference → All UI strings loaded from language JSON file via i18next → Language preference stored in user profile and localStorage → Change language option available in profile settings at any time
Priority	HIGH

3.13 User Dashboard

Description	A personalized home screen displaying the user's wallet balance, recent transactions, AI insight summary, and quick-action buttons for the most common tasks.
Inputs	User session (JWT), wallet balance, recent 5 transactions, latest AI insight summary
Outputs	Real-time balance widget, mini transaction list, insight card, quick-action buttons (Send, Receive, Invest, Insights), account status indicators
Functional Behavior	On login, dashboard fetches parallel API calls for balance, transactions, and insights → Data rendered in card-based layout optimized for mobile → Auto-refresh every 60 seconds → Animated balance transitions for new transactions

Priority	HIGH

3.14 Notifications

Description	In-app and push notification system alerting users of transaction updates, AI insights availability, security alerts, and promotional messages.
Inputs	System events (transaction completed, AI insight ready, login from new device), user notification preferences
Outputs	In-app notification bell with unread count, push notification (PWA), notification history log
Functional Behavior	Event triggers notification creation in MongoDB → WebSocket or polling delivers in-app notification → PWA service worker pushes mobile notification → Notification marked read on user interaction → User can configure notification preferences in settings
Priority	MEDIUM

3.15 Secure Data Handling

Description	All user data, financial records, and authentication tokens are handled with industry-standard encryption and security practices throughout the application stack.
Inputs	All user-submitted data, API payloads, database records
Outputs	Encrypted data at rest and in transit, secure JWT tokens, HTTPS-only communication, rate-limited APIs
Functional Behavior	All data transmitted over TLS 1.3 HTTPS → Passwords hashed with bcrypt (salt factor 12) → JWT signed with RS256 → Sensitive fields (bank account numbers, KYC documents) encrypted at rest with AES-256 → API rate limiting (100 req/min per user) → CORS policy enforced → Input sanitization on all endpoints
Priority	HIGH

4. Functional Requirements

Functional requirements are specified using the format FR-[Module].[Sub-requirement]. Each requirement is assigned a priority (High/Medium/Low) and a verifiability status.

FR-1: Authentication

Req. ID	Description	Priority
FR-1.1	The system shall allow new users to register using full name, email, mobile number, country, and preferred language.	High
FR-1.2	The system shall send a 6-digit OTP to the user's registered mobile number via Firebase Authentication.	High
FR-1.3	The OTP shall expire after 5 minutes and allow a maximum of 3 verification attempts.	High
FR-1.4	The system shall issue a signed JWT access token (expiry: 15 minutes) and a refresh token (expiry: 7 days) on successful authentication.	High
FR-1.5	The system shall support token refresh without requiring re-login while the refresh token is valid.	High
FR-1.6	The system shall invalidate all active sessions on password change or logout from all devices.	Medium
FR-1.7	The system shall detect and flag login attempts from unrecognized devices or locations.	Medium
FR-1.8	The system shall enforce a minimum password strength of 8 characters with at least 1 number and 1 special character.	High

FR-2: Wallet Management

Req. ID	Description	Priority
FR-2.1	The system shall automatically create a digital wallet for each new user upon successful registration.	High
FR-2.2	The system shall display the real-time wallet balance in the user's preferred currency.	High
FR-2.3	The system shall support a minimum of 5 currencies: USD, EUR, GBP, INR, and PHP.	High
FR-2.4	The system shall apply real-time FX rates (simulated) for cross-currency balance display.	Medium
FR-2.5	The system shall prevent any transaction that would result in a negative wallet balance.	High
FR-2.6	All wallet balance updates shall be processed as atomic operations to prevent race conditions.	High
FR-2.7	The system shall maintain a complete audit trail of all wallet balance changes.	High

FR-3: Transactions

Req. ID	Description	Priority
FR-3.1	The system shall allow a registered user to initiate a transfer to another registered user by phone number or email.	High
FR-3.2	The system shall complete a domestic transfer within 2 seconds under normal load conditions.	High
FR-3.3	The system shall apply zero service fees to all transfers.	High
FR-3.4	The system shall generate a unique transaction ID (UUID v4) for every transaction.	High
FR-3.5	The system shall generate a blockchain simulation hash (SHA-256) for every completed transaction.	Medium
FR-3.6	The system shall support transaction statuses: Initiated, Processing, Completed, Failed, Reversed.	High
FR-3.7	The system shall notify both sender and recipient upon transaction completion.	High
FR-3.8	The system shall allow users to view paginated transaction history with filtering by date, type, and status.	High
FR-3.9	The system shall automatically reverse a transaction if the recipient wallet update fails (saga pattern).	High

FR-4: AI Recommendation Engine

Req. ID	Description	Priority
FR-4.1	The system shall categorize user transactions into at least 8 expense categories using AI or rule-based classification.	High
FR-4.2	The system shall generate a personalized financial insights report when triggered by the user or on a weekly schedule.	High
FR-4.3	The system shall cache AI-generated insights for 24 hours to minimize API call costs.	Medium
FR-4.4	The system shall send anonymized and aggregated financial data to the AI API — never raw PII.	High
FR-4.5	The system shall present AI insights in the user's preferred language.	Medium
FR-4.6	The system shall suggest micro-investment products when the user's wallet balance exceeds a configurable threshold.	Medium
FR-4.7	The system shall allow users to set savings goals and track progress against AI-recommended savings amounts.	Low

FR-5: User Management

Req. ID	Description	Priority
FR-5.1	The system shall allow users to update profile information (name, profile picture, preferred language).	Medium

Req. ID	Description	Priority
FR-5.2	The system shall allow users to change their password after verifying current password.	High
FR-5.3	The system shall allow users to view their KYC status and upload supporting documents.	Medium
FR-5.4	The system shall allow users to add and manage recipient/beneficiary contacts.	Medium
FR-5.5	The system shall allow users to deactivate their account, which freezes all wallet activity.	Low

FR-6: Language Management

Req. ID	Description	Priority
FR-6.1	The system shall support the following languages: English, Hindi, Tagalog, Spanish, Arabic, Nepali, Bengali, Swahili, Urdu, and Indonesian.	High
FR-6.2	The system shall automatically detect the browser locale on first launch and suggest the matching language.	Medium
FR-6.3	The system shall persist the user's language preference in the MongoDB user profile.	High
FR-6.4	The system shall support RTL (right-to-left) text layout for Arabic and Urdu.	Medium
FR-6.5	The system shall allow language change from the profile settings page without requiring re-login.	High

FR-7: Notifications

Req. ID	Description	Priority
FR-7.1	The system shall display in-app notifications for all transaction events within 3 seconds of the event.	High
FR-7.2	The system shall support PWA push notifications for transaction and insight events.	Medium
FR-7.3	The system shall maintain a notification history log accessible from the notification bell icon.	Medium
FR-7.4	The system shall allow users to configure notification preferences (enable/disable by category).	Low
FR-7.5	The system shall mark notifications as read when the user views the notification panel.	Medium

FR-8: Security

Req. ID	Description	Priority
FR-8.1	All API endpoints shall require a valid JWT Bearer token, except /auth/register and /auth/login.	High
FR-8.2	The system shall enforce rate limiting of 100 requests per minute per user IP.	High

Req. ID	Description	Priority
FR-8.3	The system shall validate and sanitize all user inputs to prevent SQL injection and XSS attacks.	High
FR-8.4	The system shall enforce HTTPS-only communication; HTTP requests shall be redirected to HTTPS.	High
FR-8.5	The system shall log all authentication events (login, logout, failed attempts) with timestamps and IP addresses.	High
FR-8.6	The system shall lock accounts after 5 consecutive failed authentication attempts for 15 minutes.	High

FR-9: Admin Functionalities

Req. ID	Description	Priority
FR-9.1	The system shall provide an admin dashboard accessible only to users with the 'admin' role.	Medium
FR-9.2	Administrators shall be able to view, search, and filter all registered users.	Medium
FR-9.3	Administrators shall be able to suspend or reactivate user accounts.	Medium
FR-9.4	Administrators shall be able to view all platform-wide transactions with filtering capabilities.	Medium
FR-9.5	Administrators shall be able to review and approve KYC document submissions.	Low
FR-9.6	The system shall provide aggregate platform analytics (total users, transaction volume, active wallets) on the admin dashboard.	Low

5. Non-Functional Requirements

5.1 Performance

Requirement	Target / Metric
API Response Time (P95)	< 500ms under normal load (< 100 concurrent users)
Page Load Time (Initial)	< 3 seconds on 4G connection
Transaction Processing Time	< 2 seconds for domestic transfers
AI Insight Generation	< 8 seconds (includes API call to OpenAI/Gemini)
Dashboard Refresh Rate	Every 60 seconds (configurable)
Database Query Time	< 100ms for indexed queries

5.2 Scalability

- The backend shall be stateless to support horizontal scaling via container orchestration (Render.com auto-scaling)
- MongoDB Atlas shall scale vertically from M0 (free) to M10+ without application code changes
- The frontend CDN shall handle up to 100,000 concurrent requests globally through edge caching
- The API design shall support versioning (/api/v1/) to enable future breaking changes without disrupting existing clients

5.3 Availability

Component	Target SLA	Failover Strategy
Frontend	99.99%	Global CDN with edge failover
Backend API	99.5%	Auto-restart on failure, health checks
MongoDB Atlas	99.995% (M10+)	Replica set with automatic failover
Firebase Auth	99.95%	Firebase SLA guarantee
AI API (OpenAI)	99.9%	Cached responses, graceful degradation

5.4 Security

- All data in transit shall be encrypted using TLS 1.3
- All passwords shall be hashed using bcrypt with a minimum salt factor of 12
- Sensitive database fields (document IDs, bank account numbers) shall be encrypted at rest using AES-256
- JWT tokens shall use RS256 asymmetric signing algorithm
- The system shall implement OWASP Top 10 security controls
- Regular automated dependency vulnerability scanning shall be configured via npm audit

5.5 Usability

- First-time users shall be able to complete registration in under 3 minutes
- Core task flows (send money, view balance) shall require no more than 3 taps/clicks
- All error messages shall be presented in plain, user-friendly language — no technical jargon
- The UI shall comply with WCAG 2.1 Level AA accessibility guidelines
- The platform shall support screen readers (ARIA labels on all interactive elements)

5.6 Reliability

- The system shall implement idempotent transaction processing to prevent duplicate debits/credits
- The system shall implement a dead-letter queue for failed notification deliveries (retry 3 times)
- The system shall perform automated database backups every 24 hours on MongoDB Atlas
- Mean Time to Recovery (MTTR) target: < 30 minutes for critical service failures

5.7 Maintainability

- All backend code shall follow the MVC (Model-View-Controller) architectural pattern
- Code coverage from automated tests shall be maintained at a minimum of 70%
- All API endpoints shall be documented using OpenAPI 3.0 / Swagger specification
- Environment-specific configurations shall be managed via .env files and never committed to version control
- The codebase shall use ESLint and Prettier for consistent code formatting

5.8 Portability

- The frontend PWA shall be installable on Android and iOS without requiring an app store
- The backend container shall be deployable on any Docker-compatible hosting platform
- The application shall function correctly on screen sizes from 320px (small mobile) to 1920px (desktop)

5.9 Compliance & Privacy

- The system shall collect only the minimum necessary personal data (data minimization principle — GDPR Article 5)
- Users shall have the ability to request data export and deletion (GDPR Articles 15, 17)
- The Privacy Policy shall be presented and accepted during registration
- All KYC data shall be stored in a segregated, access-controlled collection
- System logs shall not contain PII (personally identifiable information)

6. System Architecture

FinWave follows a modern three-tier, cloud-native architecture separating presentation, application logic, and data concerns.

6.1 Frontend Architecture

The frontend is built as a React Progressive Web Application (PWA) styled with Tailwind CSS, structured using a component-based, feature-first folder organization:

Layer	Technology	Responsibility
Routing	React Router v6	Client-side navigation, protected route guards
State Management	Redux Toolkit / Context API	Global auth state, wallet balance, notifications
Data Fetching	Axios + React Query	API calls, caching, background refetching
Internationalization	react-i18next	Language loading, locale switching, RTL support
Styling	Tailwind CSS	Utility-first responsive design, mobile-first breakpoints
Charts	Recharts / Chart.js	Expense category pie charts, spending trend line charts
PWA	Service Worker	Offline caching, push notifications, install prompt

6.2 Backend Architecture

The backend follows a RESTful MVC pattern implemented in Node.js with Express.js, organized into clearly separated layers:

- Routes Layer: Express.js route definitions with middleware chains (auth, validation, rate-limit)
- Controller Layer: Business logic handlers, request/response orchestration
- Service Layer: Core domain logic (wallet, transaction processing, AI orchestration, blockchain simulation)
- Model Layer: Mongoose ODM schemas, validation, and pre-save hooks
- Middleware Layer: JWT verification, error handling, CORS, helmet security headers, request logging

6.3 Database Architecture

MongoDB Atlas is used as the primary datastore with a document-oriented schema optimized for the read-heavy, event-driven nature of financial applications. Collections are designed to minimize joins (embedding where appropriate) while maintaining data integrity through application-level transactions.

6.4 API Flow

Client → HTTPS Request → CDN (static) or Render.com API → Express.js Router → Middleware (JWT Verify → Rate Limit → Validate) → Controller → Service → Mongoose Model → MongoDB Atlas → Response serialized → JSON returned to client.

6.5 Authentication Flow

Step 1: User submits phone number → Step 2: Firebase reCAPTCHA verify → Step 3: Firebase sends SMS OTP → Step 4: User submits OTP → Step 5: Firebase verifies OTP, returns Firebase ID token → Step 6:

Client sends Firebase ID token to backend /auth/firebase → Step 7: Backend verifies Firebase token with Firebase Admin SDK → Step 8: Backend locates or creates user in MongoDB → Step 9: Backend issues application JWT (RS256) + refresh token → Step 10: Client stores tokens securely (httpOnly cookie or memory).

6.6 AI Integration Flow

Step 1: User triggers 'Generate Insights' → Step 2: Backend checks Redis/MongoDB cache (24h TTL) → Step 3 (cache miss): Backend aggregates anonymized transaction data from MongoDB → Step 4: Backend constructs structured prompt with spending summary and user profile → Step 5: POST to OpenAI/Gemini API → Step 6: AI returns structured JSON response → Step 7: Backend parses, validates, and stores in AI_Insights collection → Step 8: Response returned to frontend → Step 9: Frontend renders insight cards with charts.

6.7 Simulated Blockchain Flow

Step 1: Transaction marked as 'Completed' in MongoDB → Step 2: Blockchain simulation service triggered → Step 3: Transaction data (sender, receiver, amount, currency, timestamp, previous hash) serialized → Step 4: SHA-256 hash computed using Node.js crypto module → Step 5: Block object created {blockIndex, timestamp, data, previousHash, hash} → Step 6: Block appended to Blockchain collection → Step 7: Transaction document updated with blockHash field → Step 8: 'Verified' badge displayed to user.

6.8 Cloud Deployment Architecture

Component	Platform	Configuration
React PWA	AWS	Auto-deploy from GitHub main branch, global CDN, custom domain SSL
Node.js API	AWS	Web Service (free tier), auto-scaling, health check endpoint
MongoDB	MongoDB Atlas	M0 (free) / M10 (production), VPC peering, IP whitelist
Firebase Auth	Google Firebase	Phone Auth enabled, free Spark plan for prototype
Environment Config	.env on Render	All secrets managed via Render environment variables
CI/CD	GitHub Actions	Automated lint, test, and deploy pipeline on PR merge

7. Database Design

FinWave uses MongoDB Atlas with five primary collections. Document structures are optimized for common query patterns in a fintech context.

7.1 Users Collection

Field	Type	Description	Constraints
_id	ObjectId	MongoDB auto-generated unique identifier	Primary Key, Indexed
firebaseUID	String	Firebase Authentication UID	Unique, Indexed, Required
fullName	String	User's legal full name	Required, 2–100 chars
email	String	User's email address	Unique, Indexed, Required
phoneNumber	String	Verified mobile number (E.164 format)	Unique, Indexed, Required
country	String	ISO 3166-1 alpha-2 country code	Required
preferredLanguage	String	BCP 47 language tag (e.g., 'en', 'hi')	Required, Default: 'en'
role	String	User role enum: 'user', 'admin'	Default: 'user'
kycStatus	String	KYC verification status: pending/verified/rejected	Default: 'pending'
isActive	Boolean	Account active status	Default: true
createdAt	Date	Account creation timestamp	Auto-set
updatedAt	Date	Last profile update timestamp	Auto-updated

7.2 Wallets Collection

Field	Type	Description	Constraints
_id	ObjectId	Wallet unique identifier	Primary Key
userId	ObjectId	Reference to Users._id	Foreign Key, Indexed, Unique
walletId	String	Human-readable wallet ID (UUID)	Unique, Indexed
balance	Decimal128	Current wallet balance	Min: 0, Required
currency	String	ISO 4217 currency code (e.g., USD)	Required, Default: 'USD'
isLocked	Boolean	Wallet freeze status for security holds	Default: false
lastActivityAt	Date	Timestamp of last transaction	Auto-updated
createdAt	Date	Wallet creation timestamp	Auto-set

7.3 Transactions Collection

Field	Type	Description	Constraints
_id	ObjectId	Transaction unique identifier	Primary Key
transactionId	String	UUID v4 transaction reference	Unique, Indexed
senderId	ObjectId	Reference to Users._id (sender)	Indexed, Required
receiverId	ObjectId	Reference to Users._id (receiver)	Indexed, Required
amount	Decimal128	Transaction amount in source currency	Min: 0.01, Required
currency	String	ISO 4217 source currency code	Required
destinationAmount	Decimal128	Amount in destination currency	Computed
destinationCurrency	String	ISO 4217 destination currency code	Required
fxRate	Number	Applied foreign exchange rate	Computed
fee	Number	Service fee (always 0 in FinWave)	Default: 0
status	String	Enum: initiated/processing/completed/failed/reversed	Required
type	String	Enum: transfer/investment/deposit/withdrawal	Required
blockchainHash	String	SHA-256 hash from blockchain simulation	Optional
note	String	Optional sender message	Max 200 chars
createdAt	Date	Transaction initiation timestamp	Auto-set, Indexed

7.4 AI_Insights Collection

Field	Type	Description	Constraints
_id	ObjectId	Insight document identifier	Primary Key
userId	ObjectId	Reference to Users._id	Indexed, Required
generatedAt	Date	Timestamp of AI insight generation	Indexed
expiresAt	Date	Cache expiry (24h from generation)	TTL Index
spendingBreakdown	Object	Category-wise expense totals	JSON Object
savingsRecommendation	String	AI-generated savings tip	Max 500 chars
investmentSuggestions	Array	List of recommended investment products	Array of Objects
riskScore	Number	User financial risk score (1–10)	Computed by AI
insightText	String	Full plain-language insight paragraph	Max 1000 chars
model	String	AI model used (e.g., 'gpt-4o-mini')	Required

7.5 Notifications Collection

Field	Type	Description	Constraints
_id	ObjectId	Notification identifier	Primary Key
userId	ObjectId	Reference to Users._id (recipient)	Indexed, Required
type	String	Enum: transaction/insight/security/system/promo	Required
title	String	Notification title	Max 100 chars
body	String	Notification message body	Max 500 chars
isRead	Boolean	Read status	Default: false
relatedId	ObjectId	Reference to related entity (transaction, insight)	Optional
createdAt	Date	Notification creation timestamp	Auto-set, Indexed

8. API Design

All API endpoints are prefixed with /api/v1/ and return JSON. Authentication endpoints are public; all others require a valid JWT Bearer token in the Authorization header. The full API is documented in OpenAPI 3.0 format.

8.1 Authentication — Register

	Registers a new user account. Firebase OTP verification must be completed before calling this endpoint.
Request Body	`{ "fullName": "Juan dela Cruz", "email": "juan@example.com", "phoneNumber": "+639171234567", "firebaseToken": "eyJ...", "country": "PH", "preferredLanguage": "tl" }`
Response Body	`{ "success": true, "message": "Account created successfully", "data": { "userId": "64abc...", "walletId": "wlt_...", "accessToken": "eyJ...", "refreshToken": "eyJ..." } }`
Status Codes	`201 Created | 400 Bad Request | 409 Conflict (email/phone exists) | 401 Unauthorized (invalid Firebase token)`

8.2 Authentication — Login

	Authenticates an existing user. Client must obtain Firebase OTP token first and pass it for backend verification.
Request Body	`{ "phoneNumber": "+639171234567", "firebaseToken": "eyJ..." }`
Response Body	`{ "success": true, "data": { "userId": "64abc...", "accessToken": "eyJ...", "refreshToken": "eyJ...", "expiresIn": 900 } }`
Status Codes	`200 OK | 401 Unauthorized | 403 Forbidden (account suspended) | 429 Too Many Requests`

8.3 Wallet — Fetch Balance

	Returns the authenticated user's wallet balance and currency. Requires Bearer token.
Request Body	`N/A (no body required)`
Response Body	`{ "success": true, "data": { "walletId": "wlt_abc123", "balance": "1250.75", "currency": "USD", "isLocked": false, "lastActivityAt": "2026-05-22T10:30:00Z" } }`
Status Codes	`200 OK | 401 Unauthorized | 404 Not Found`

8.4 Transactions — Send Money

	Initiates a zero-fee money transfer to another registered FinWave user. Atomically debits sender and credits receiver.

Request Body	`{ "recipientIdentifier": "+919876543210", "amount": 100.00, "currency": "USD", "destinationCurrency": "INR", "note": "Monthly support" }`
Response Body	`{ "success": true, "data": { "transactionId": "txn_...", "status": "completed", "blockchainHash": "a3f9bc...", "senderBalance": "1150.75", "fxRate": 83.45, "destinationAmount": 8345.00, "createdAt": "2026-05-22T10:35:00Z" } }`
Status Codes	`201 Created | 400 Bad Request | 402 Insufficient Funds | 404 Recipient Not Found | 422 Unprocessable Entity`

8.5 Transactions — Get History

	Returns paginated transaction history for the authenticated user with optional filters.
Request Body	`N/A (no body required)`
Response Body	`{ "success": true, "data": { "transactions": [...], "pagination": { "page": 1, "limit": 20, "total": 87, "pages": 5 } } }`
Status Codes	`200 OK | 401 Unauthorized | 422 Invalid Filter Parameters`

8.6 AI Insights — Generate

	Triggers AI insight generation for the authenticated user. Returns cached result if available (24h TTL).
Request Body	`{ "forceRefresh": false }`
Response Body	`{ "success": true, "data": { "insightId": "ins_...", "spendingBreakdown": { "remittance": 450, "food": 120, "transport": 80 }, "savingsRecommendation": "You can save $75 more per month...", "riskScore": 4, "insightText": "...", "generatedAt": "2026-05-22T...", "isCached": true } }`
Status Codes	`200 OK | 401 Unauthorized | 503 AI Service Unavailable`

8.7 User — Language Settings

	Updates the authenticated user's preferred language. Changes take effect immediately on next page render.
Request Body	`{ "language": "hi" }`
Response Body	`{ "success": true, "message": "Language updated to Hindi", "data": { "preferredLanguage": "hi" } }`
Status Codes	`200 OK | 400 Bad Request (unsupported language) | 401 Unauthorized`

9. UI/UX Requirements

9.1 Mobile-First Design

The FinWave interface is designed with a mobile-first philosophy, targeting smartphones as the primary device. All layouts are constructed using Tailwind CSS mobile breakpoints (default: 320px), progressively enhanced for tablets (md: 768px) and desktops (lg: 1024px+). Touch targets are minimum 44x44px per Apple HIG and Google Material Design guidelines.

9.2 Navigation Structure

The primary navigation uses a bottom tab bar (mobile) / left sidebar (desktop) with five core tabs: Home (Dashboard), Send Money, Wallet, Insights, and Profile. This pattern is familiar to mobile banking app users and minimizes cognitive load for first-time users.

9.3 Accessibility Standards

- WCAG 2.1 Level AA compliance throughout all screens
- ARIA labels on all buttons, form fields, and interactive elements
- Color contrast ratio minimum 4.5:1 for normal text, 3:1 for large text
- Keyboard navigable (Tab key focus order follows logical visual flow)
- Screen reader compatible (tested with VoiceOver and TalkBack)
- Text scaling support: UI remains functional at 200% browser zoom

9.4 Low-Tech User Support

Given the target demographic of migrant workers who may have limited smartphone literacy, FinWave incorporates several low-tech user support measures:

- Icon + Label navigation: Every button displays both an icon and a text label
- Progressive disclosure: Advanced features hidden until user is comfortable with basics
- Contextual help tooltips: '?' icon adjacent to complex financial terms
- One-tap QR code sharing for receiving payments (eliminates manual entry)
- Voice input support on all text fields (leverages native device dictation)
- Simplified mode: Toggle that hides investment and advanced AI features for basic users

9.5 Multilingual Interface

- All UI strings externalized to JSON language files (no hardcoded strings in JSX)
- Language selection available on the login screen and profile settings
- RTL layout automatically applied for Arabic and Urdu
- Number and currency formatting localized using Intl.NumberFormat API
- Date and time display formatted per locale using Intl.DateTimeFormat

9.6 Design System

Element	Specification
Primary Color	#1B4F72 (Deep Navy Blue) — Trust, financial stability
Accent Color	#1ABC9C (Teal) — Action, success, positive sentiment
Typography	Inter (UI), Noto Sans (Multilingual fallback) — Both Google Fonts
Border Radius	12px for cards, 8px for inputs, 24px for primary buttons
Spacing System	4px base unit, multiples of 4 (8, 12, 16, 24, 32, 48px)
Shadow	Subtle card shadow: 0 2px 8px rgba(0,0,0,0.08) — light and modern
Icon Library	Lucide React — consistent, open-source, tree-shakeable

10. Security Requirements

10.1 Encryption

- All client-server communication enforced over TLS 1.3 (minimum TLS 1.2)
- Passwords hashed using bcrypt with salt rounds = 12 before storage
- Sensitive document fields (KYC data) encrypted at rest using AES-256-CBC
- Encryption keys managed via environment variables, never stored in code
- JWT tokens signed using RS256 (RSA-SHA256) asymmetric keys, private key stored server-side only

10.2 Firebase Authentication

- Firebase Phone Auth handles OTP generation, delivery, and verification — reducing PII exposure on the backend
- Firebase reCAPTCHA v3 used to prevent automated bot registrations
- Firebase ID tokens verified server-side using Firebase Admin SDK on every protected API call
- Firebase sessions are token-based; no traditional cookie sessions maintained

10.3 JWT / Session Handling

- Access Token lifetime: 15 minutes (short-lived to minimize exposure window)
- Refresh Token lifetime: 7 days, stored securely in httpOnly cookies
- Token rotation: New refresh token issued on every use
- Token revocation list maintained in MongoDB for logout/compromise scenarios
- CSRF protection: SameSite=Strict cookie attribute + custom header validation

10.4 API Security

- All endpoints protected by Helmet.js middleware (sets 11 HTTP security headers)
- CORS configured to allow only the registered frontend domain
- Rate limiting: 100 requests/minute per IP (express-rate-limit)
- Input validation using express-validator on all request bodies
- SQL/NoSQL injection prevention via mongoose query sanitization
- express-mongo-sanitize removes $ and . characters from user inputs

10.5 KYC Concepts

While full KYC integration is beyond the hackathon scope, the platform is architected to support KYC compliance:

- User identity document upload fields included in the data model
- KYC status field (pending/verified/rejected) gates access to higher transaction limits
- Admin interface for reviewing uploaded KYC documents
- In production: integration with third-party KYC providers (Jumio, Onfido, or Digilocker for India)

10.6 Fraud Prevention

- Velocity checks: Flag if a single user initiates more than 10 transfers within 1 hour
- Amount anomaly detection: Alert if transfer amount is 3x user's average transaction
- Device fingerprinting: New device login triggers notification and optional re-verification
- Blacklist: Phone numbers and emails flagged for fraud are blocked from registration
- Transaction cooling period: Transfers above configurable threshold require confirmation delay

11. AI Module Description

The AI module is the intelligence layer of FinWave, transforming raw transaction data into actionable financial guidance. It leverages the OpenAI GPT-4o-mini or Google Gemini 1.5 Flash API.

11.1 Expense Analysis

The expense analysis engine automatically categorizes each transaction into one of 8 primary categories (Remittance, Food & Groceries, Housing & Rent, Transportation, Healthcare, Leisure & Entertainment, Savings & Investments, Other) using a hybrid approach:

- Rule-based classification: Transactions to known wallets (remittance) or merchants (food apps) are auto-tagged
- AI-based classification: Ambiguous transactions described in natural language are classified by the LLM
- Category totals aggregated monthly and displayed as an interactive pie chart

11.2 Savings Recommendations

The savings recommendation engine analyzes income patterns and discretionary spending to suggest realistic savings targets. The AI prompt includes:

- Monthly income estimate (derived from recurring credits)
- Average discretionary spending per category
- User-stated goals (if available)
- Country of residence and destination (cost-of-living context)

The AI returns a structured recommendation including: monthly savings target ($), percentage of income to save, and a 3-step savings plan in plain language.

11.3 Personalized Financial Insights

Weekly personalized insights are generated by passing the following anonymized data to the AI API:

Input to AI	Purpose
Spending category totals (current vs. previous month)	Trend detection — spending more on food? transport?
Remittance frequency and amount	Identify optimal remittance timing for best FX rates
Wallet balance trend (7-day)	Cash flow health indicator
User's country of residence and origin	Cultural and cost-of-living context
User's preferred language	Localized, culturally relevant advice

11.4 Micro-Investment Suggestions

When wallet balance exceeds a user-configurable threshold (default: $100 surplus), the AI module suggests 3 simulated micro-investment products: a low-risk bond fund, a gold ETF unit, and a balanced equity fund. Each suggestion includes expected annual return (%), risk rating (1–5), minimum investment amount, and a plain-language explanation of the product's risk profile.

11.5 Future AI Enhancements

- Credit scoring: AI-derived creditworthiness score based on transaction behavior for micro-loan access
- Fraud detection: Real-time transaction anomaly scoring using ML model
- Voice assistant: Multilingual voice interface for balance queries and send-money commands
- Predictive remittance: AI predicts optimal times to send money based on FX rate forecasting

12. Blockchain Simulation Module

The Blockchain Simulation Module provides a trust and transparency mechanism for FinWave transactions without the complexity and cost of a real distributed blockchain network. It simulates the core concepts of blockchain — cryptographic hashing, chain linkage, and immutability — using Node.js's built-in crypto module and MongoDB.

12.1 Simulated Ledger

The simulated blockchain is stored in a dedicated 'Blockchain' MongoDB collection. Each entry represents a 'block' in the chain, containing:

Field	Value / Purpose
blockIndex	Sequential integer (Genesis block = 0)
timestamp	ISO 8601 UTC timestamp of block creation
transactionData	JSON object with sender, receiver, amount, currency, transactionId
previousHash	SHA-256 hash of the previous block (chain linkage)
hash	SHA-256 hash of current block's contents
nonce	Simple counter simulating proof-of-work difficulty

12.2 Transaction Hashing

The hashing algorithm uses Node.js's native crypto.createHash('sha256') function. The block data is serialized as a deterministic JSON string (keys sorted alphabetically) before hashing to ensure consistent hash values regardless of object key insertion order.

Block Hash = SHA-256(blockIndex + timestamp + JSON.stringify(sortedData) + previousHash + nonce)

12.3 Verification Mechanism

Users can verify any transaction by:

6. Copying the blockchain hash from the transaction detail screen
7. Tapping 'Verify on Chain' button — the frontend calls GET /api/v1/blockchain/verify/:hash
8. The backend retrieves the block, recomputes the hash from stored data, and compares with the stored hash
9. A green 'Verified' badge with timestamp is displayed to the user if hashes match

12.4 Transparency Concept

The blockchain simulation demonstrates the concept of financial transparency to users who may distrust traditional financial institutions. By giving each transaction a verifiable hash that users can independently check, the platform builds trust. In a production version, this simulation would be replaced with a real permissioned blockchain (e.g., Hyperledger Fabric or Polygon) or integrated with an existing regulated DeFi infrastructure.

13. Cloud & Deployment

13.1 AWS Amplify — Frontend Deployment

- GitHub repository connected to AWS Amplify via GitHub App integration
- Automatic preview deployments generated for every pull request
- Production deployment triggered on merge to 'main' branch
- Global CDN with 30+ edge regions ensures < 100ms TTFB worldwide
- Custom domain with automatic SSL certificate provisioning
- Environment variables (REACT_APP_API_URL, REACT_APP_FIREBASE_CONFIG) configured in AWS dashboard

13.2 AWS EC2 — Backend Hosting

- Node.js web service deployed as a Docker container on Render.com
- Auto-deploy on push to 'main' branch via GitHub webhook
- Health check endpoint at /api/health — Render restarts the service if health check fails
- Environment variables (MONGODB_URI, JWT_PRIVATE_KEY, OPENAI_API_KEY, FIREBASE_SERVICE_ACCOUNT) secured in Render dashboard
- Free tier: Single instance, 512MB RAM, spun down after 15 minutes of inactivity (upgrade for production)

13.3 MongoDB Atlas

- M0 free cluster used for hackathon prototype (512MB storage, shared CPU)
- M10 dedicated cluster recommended for production ($57/month, 2GB RAM, 10GB storage)
- IP Access List configured to allow only Render.com outbound IP ranges
- Database user with least-privilege access (read/write on FinWave DB only)
- Automated daily backups enabled (M10+), point-in-time recovery available
- MongoDB Atlas Search (Lucene-based) available for transaction full-text search in future

13.4 CI/CD Pipeline

Stage	Tool	Action
Lint	ESLint + Prettier	Enforce code style and catch syntax errors
Test	Jest + Supertest	Run unit and integration tests, minimum 70% coverage
Build	Vite (frontend)	Production bundle optimization and tree-shaking
Deploy Preview	AWS	Deploy to preview URL on PR creation
Deploy Production	AWS	Auto-deploy on merge to main after tests pass
Security Scan	npm audit + Snyk	Dependency vulnerability check on every build

13.5 Scalability Approach

The stateless backend design allows horizontal scaling by adding more autoscaling instances behind a load balancer. MongoDB Atlas scales vertically (larger instance) or with horizontal sharding for massive data volumes. AWS Cloudfront edge CDN scales automatically. For the hackathon prototype, a single backend instance is sufficient. Production architecture would include Redis for session caching and rate limit state sharing across instances.

14. Use Case Diagrams (Textual Explanation)

14.1 Actor Definitions

Actor	Description
Registered User (Migrant Worker)	Primary actor. Can perform all core financial operations.
Guest User	Limited actor. Can only view landing page and initiate registration.
Recipient	Secondary actor. Receives money; may or may not be a registered user.
System (AI Engine)	Automated actor. Generates insights on schedule or on demand.
Administrator	Privileged actor. Manages users, reviews KYC, views analytics.
Firebase Auth	External actor. Handles OTP generation and verification.

14.2 Use Case: User Login

Actor: Registered User | Precondition: User has a registered account

- UC-1: User opens FinWave and taps 'Login'
- UC-1.1: User enters registered phone number → System displays OTP request
- UC-1.2: Firebase Auth sends 6-digit OTP via SMS
- UC-1.3: User enters OTP → Firebase verifies → Backend issues JWT
- UC-1.4 (Alternative): User enters wrong OTP → System shows error, allows 2 more attempts
- UC-1.5 (Exception): OTP expires → System prompts user to request new OTP

14.3 Use Case: Send Money

Actor: Registered User | Precondition: User is authenticated, wallet has sufficient balance

- UC-2: User taps 'Send Money' from dashboard
- UC-2.1: User enters recipient identifier (phone/email) or scans QR code
- UC-2.2: System validates recipient exists and is active
- UC-2.3: User enters amount and selects destination currency → System shows FX preview
- UC-2.4: User reviews and confirms → System processes transfer atomically
- UC-2.5: Blockchain hash generated → Both parties notified → Transaction recorded
- UC-2.6 (Alternative): Insufficient balance → System shows error with current balance

14.4 Use Case: Receive Money

Actor: Registered User | Precondition: Sender initiates transfer to this user's identifier

- UC-3: System receives credit event from transaction service
- UC-3.1: Recipient's wallet balance updated atomically
- UC-3.2: In-app and push notification dispatched to recipient
- UC-3.3: Transaction record created in recipient's history with 'Credit' type

14.5 Use Case: View AI Insights

Actor: Registered User | Precondition: User has at least 5 transactions in the last 30 days

- UC-4: User navigates to 'Insights' tab
- UC-4.1: System checks cache — if valid (< 24h old), returns cached insights
- UC-4.2 (Cache miss): System aggregates transaction data → Sends structured prompt to AI API
- UC-4.3: AI API returns analysis → System parses, stores, and returns to frontend
- UC-4.4: Frontend renders spending chart, savings tip, and investment suggestion cards

14.6 Use Case: Change Language

Actor: Registered User | Precondition: User is authenticated

- UC-5: User navigates to Profile → Language Settings
- UC-5.1: User selects desired language from dropdown (10+ options)
- UC-5.2: System updates user profile (PUT /api/v1/user/language)
- UC-5.3: react-i18next reloads language JSON → All UI strings update instantly
- UC-5.4: RTL layout toggled if Arabic or Urdu selected

14.7 Use Case: Track Transactions

Actor: Registered User | Precondition: User is authenticated

- UC-6: User navigates to 'History' tab
- UC-6.1: System fetches first page (20 items) of transactions sorted by date descending
- UC-6.2: User applies filters (date range, type, status) → System refetches with new query params
- UC-6.3: User taps a transaction → Detail screen shows amount, parties, hash, timestamp, status
- UC-6.4: User taps 'Verify on Chain' → System validates blockchain hash → Shows verification badge

15. Activity Diagrams (Textual Explanation)

15.1 User Registration Activity Flow

[START] → User opens FinWave → User taps 'Create Account' → User fills registration form → [Decision: Inputs valid?] → [No] → Show validation errors → Return to form → [Yes] → System calls Firebase Auth phone number enroll → Firebase sends OTP SMS → User enters OTP → [Decision: OTP correct?] → [No, attempts < 3] → Show error → Re-enter OTP → [No, attempts = 3] → Lock for 5 minutes → [Yes] → Backend creates User document → Backend creates Wallet document → System sends Welcome notification → JWT issued → User redirected to Dashboard → [END]

15.2 Zero-Fee Money Transfer Activity Flow

[START] → User authenticated on Dashboard → User taps 'Send Money' → [Decision: Enter recipient manually OR scan QR?] → [QR] → Camera opens → QR decoded → Pre-fill recipient → [Manual] → User types phone/email → System looks up recipient → [Decision: Recipient found?] → [No] → Show error → Return → [Yes] → User enters amount → User selects destination currency → System computes FX rate and destination amount → User reviews summary screen → [Decision: User confirms?] → [No] → Cancel → [Yes] → System validates sender balance → [Decision: Balance sufficient?] → [No] → Show insufficient funds error → [Yes] → Begin atomic transaction (MongoDB session) → Debit sender wallet → Credit receiver wallet → [Decision: Both operations successful?] → [No] → Rollback both wallet updates → Record failed transaction → Notify sender of failure → [Yes] → Commit transaction → Blockchain simulation generates hash → Record transaction document → Notify sender (success + receipt) → Notify receiver (credit received) → Update dashboards → [END]

15.3 AI Insight Generation Activity Flow

[START] → User taps 'Generate Insights' OR weekly scheduler triggers → System checks MongoDB AI_Insights cache for userId with expiresAt > now → [Decision: Valid cache exists?] → [Yes] → Return cached insights → Render frontend cards → [END] → [No] → System queries Transactions collection for last 90 days → Aggregate by category (8 buckets) → Compute monthly totals and trends → Construct AI prompt: { userData: anonymized spending summary, language: user preference, goals: user goals if set } → POST to OpenAI/Gemini API → [Decision: API response received?] → [No, error/timeout] → Return graceful degradation message 'Insights temporarily unavailable' → [Yes] → Parse AI JSON response → Validate response schema → Store new AI_Insights document with 24h TTL → Update transaction records with AI category tags → Return insights to frontend → Render spending chart + savings card + investment suggestions → [END]

15.4 Blockchain Verification Activity Flow

[START] → Transaction completed → Blockchain service triggered → Retrieve previous block hash from Blockchain collection (or genesis hash '0000...') → Construct block object { blockIndex, timestamp, transactionData, previousHash } → Serialize data as sorted JSON string → Compute SHA-256 hash → Increment nonce and recompute if hash doesn't meet difficulty criteria (simulated PoW) → Store block in Blockchain collection → Update Transaction document with blockHash field → Set transaction verification status to 'Verified' → User views transaction → User taps 'Verify' → Backend retrieves block → Recomputes hash from stored data → Compares computed hash with stored hash → [Match?] → [Yes] → Return 'Verified' status with block timestamp → Display green verification badge → [No] → Return 'Tampered/Invalid' status → Alert admin → [END]

16. Sequence Diagrams (Textual Explanation)

Sequence diagrams describe the chronological message exchange between system components for key scenarios.

16.1 User Authentication Sequence

Step	From	To	Message / Action
1	User (Browser)	React Frontend	Enter phone number, tap Send OTP
2	React Frontend	Firebase Auth SDK	auth.signInWithPhoneNumber(phone, appVerifier)
3	Firebase Auth SDK	Firebase Servers	Request OTP generation for phone
4	Firebase Servers	User's Mobile (SMS)	Deliver 6-digit OTP
5	User (Browser)	React Frontend	Enter OTP, tap Verify
6	React Frontend	Firebase Auth SDK	confirmationResult.confirm(otp)
7	Firebase Auth SDK	Firebase Servers	Verify OTP against stored challenge
8	Firebase Servers	Firebase Auth SDK	Return Firebase ID token
9	React Frontend	Node.js API	POST /auth/login { firebaseToken }
10	Node.js API	Firebase Admin SDK	admin.auth().verifyIdToken(firebaseToken)
11	Firebase Admin SDK	Node.js API	Return decoded token { uid, phone }
12	Node.js API	MongoDB Atlas	findOne({ firebaseUID: uid })
13	MongoDB Atlas	Node.js API	Return User document
14	Node.js API	React Frontend	Return { accessToken, refreshToken, userId }
15	React Frontend	User (Browser)	Store tokens, redirect to Dashboard

16.2 Money Transfer Sequence

Step	From	To	Message / Action
1	User	React Frontend	Submit transfer form { recipient, amount, currency }
2	React Frontend	Node.js API	POST /transactions/send (Bearer token in header)
3	Node.js API	JWT Middleware	Verify access token signature and expiry
4	JWT Middleware	Node.js API	Token valid → proceed with userId
5	Node.js API	MongoDB Atlas	Find recipient by phone/email
6	MongoDB Atlas	Node.js API	Return recipient User document
7	Node.js API	MongoDB Atlas	Start MongoDB session → debit sender wallet
8	Node.js API	MongoDB Atlas	Credit recipient wallet (same session)
9	MongoDB Atlas	Node.js API	Both updates committed → success

Step	From	To	Message / Action
10	Node.js API	Blockchain Service	generateBlock(transactionData)
11	Blockchain Service	MongoDB Atlas	Store block → return hash
12	Node.js API	MongoDB Atlas	Create Transaction document with hash
13	Node.js API	Notification Service	Emit events for sender + recipient notifications
14	Node.js API	React Frontend	Return 201 { transactionId, hash, status }
15	React Frontend	User	Display success screen with transaction receipt

16.3 AI Insight Generation Sequence

Step	From	To	Message / Action
1	User	React Frontend	Tap 'Generate Insights' button
2	React Frontend	Node.js API	POST /ai/insights { forceRefresh: false }
3	Node.js API	MongoDB Atlas	Query AI_Insights where userId = X, expiresAt > now
4a (Cache HIT)	MongoDB Atlas	Node.js API	Return cached insight document
4b (Cache MISS)	MongoDB Atlas	Node.js API	Return null / empty
5b	Node.js API	MongoDB Atlas	Aggregate transactions (last 90 days) by category
6b	MongoDB Atlas	Node.js API	Return aggregated spending summary
7b	Node.js API	OpenAI/Gemini API	POST /v1/messages { prompt with spending data }
8b	OpenAI/Gemini API	Node.js API	Return structured insights JSON
9b	Node.js API	MongoDB Atlas	Store new AI_Insights document (TTL 24h)
10	Node.js API	React Frontend	Return insight object { breakdown, savings, risk }
11	React Frontend	User	Render spending pie chart, savings card, investment cards

17. Future Scope

The following enhancements are planned for post-hackathon development phases:

Feature	Phase	Description
Real Banking Integration	Phase 2	Connect to banking APIs (Plaid, Open Banking UK) for real account-to-account transfers and balance sync
UPI Integration (India)	Phase 2	Integrate with NPCI's UPI rails for zero-fee INR transfers within India
Real Blockchain (Polygon)	Phase 3	Replace simulated ledger with Polygon PoS blockchain for true immutable transaction records
International Remittance Rails	Phase 2	Partner with licensed payment processors (dLocal, Thunes) for real cross-border settlement
Micro-Insurance Products	Phase 3	Offer parametric insurance (health, travel, income protection) via API partners (Bima, Inclusivity Solutions)
Micro-Loan System	Phase 3	AI credit scoring-based micro-loans ($10–$500) for emergency needs, repaid via wallet
AI Credit Scoring	Phase 3	Machine learning model trained on transaction behavior to generate alternative credit scores for unbanked users
Voice Assistant	Phase 3	Multilingual voice interface for balance queries, send-money commands, and financial Q&A
Offline Mode	Phase 2	PWA offline capability with sync queue for balance viewing and transfer initiation in low-connectivity areas
Group Savings (Chama)	Phase 3	Digital Chama/ROSCA — group savings pools popular in African and South Asian migrant communities
Merchant Payments	Phase 4	QR code-based payments at local merchants, bill payments, and mobile top-ups
Family Dashboard	Phase 4	Shared view allowing family members in home country to view incoming transfers and account activity

18. Advantages of the System

Advantage	Details
Zero Transfer Fees	Eliminates the 5–10% remittance tax that costs migrant workers billions annually. Every dollar saved goes directly to families.
Financial Inclusion	Provides banking-equivalent services to the unbanked and underbanked without requiring a traditional bank account.
AI-Powered Guidance	Democratizes access to financial advice previously available only to affluent bank customers through personalized AI coaching.
Multilingual Accessibility	10+ language support removes the language barrier that prevents many migrants from fully utilizing financial platforms.
Trust Through Transparency	Blockchain verification gives users a verifiable audit trail, building trust in a population historically exploited by informal money transfer operators.
Mobile-First Design	Optimized for the devices and connectivity levels of the target demographic, ensuring usability in real-world conditions.
Low Barrier to Entry	No minimum balance, no monthly fees, no credit check required for basic wallet and remittance features.
Privacy-Preserving AI	AI insights generated from anonymized aggregated data — no raw PII sent to third-party AI providers.
Scalable Architecture	Cloud-native, stateless backend designed to scale from a hackathon prototype to millions of users with infrastructure changes only.
Open for Integration	REST API design enables future integration with local banking systems, UPI, mobile money (M-Pesa), and global remittance rails.

19. Limitations of the Current Prototype

Limitation	Impact	Mitigation Path
Simulated Transactions	No real money moves; transfers are balance updates in MongoDB only	Integrate licensed payment processor (Phase 2)
Simulated Blockchain	Not a real distributed ledger; security guarantees are illustrative	Migrate to Polygon or Hyperledger Fabric (Phase 3)
No Real KYC	Identity verification is cosmetic; real AML/KYC compliance not implemented	Integrate Jumio or Onfido (Phase 2)
AI Rate Limits	OpenAI/Gemini API has rate limits; demo may throttle under high load	Implement request queuing and better caching
Limited Currencies	Only 5 currencies supported; FX rates are simulated, not live	Integrate Open Exchange Rates API (Phase 2)
Free-Tier Backend	Render.com free tier sleeps after 15 minutes; cold start latency ~30s	Upgrade to paid instance for production
No Push Notifications	PWA push notifications require production service worker setup; in-app only in demo	Complete service worker implementation (Phase 2)
No Offline Mode	Application requires internet connection for all operations	Implement Service Worker cache strategies (Phase 2)
Limited Test Coverage	Hackathon timeline limits automated test coverage below 70% target	Expand test suite post-hackathon
Single-Region Deployment	Backend hosted in a single Render.com region; no multi-region failover	Deploy to multiple regions with load balancer (Phase 3)

20. Conclusion

FinWave — the FinTech Financial Inclusion Platform for Migrant Workers — represents a technically ambitious, socially impactful, and architecturally sound solution to one of the most significant financial equity challenges of our time. The platform addresses the intersection of financial exclusion, language barriers, high remittance costs, and lack of investment access that disproportionately affects the global migrant worker population.

This Software Requirements Specification has documented, in comprehensive IEEE-standard detail, every aspect of the system — from user-facing features and functional requirements, to security architecture, database schemas, API contracts, AI integration patterns, and blockchain simulation mechanisms. The specification is intended to serve not only as a development reference but as a testament to the engineering rigor and social purpose behind Team 20's hackathon submission.

The prototype demonstrates that with modern cloud-native technology — React, Node.js, MongoDB Atlas, Firebase Authentication, and the OpenAI API — a team can build a functional, secure, and user-centered financial inclusion platform within the compressed timeline of a hackathon. The architecture is intentionally designed for extensibility, with clear pathways to production-grade banking integrations, real blockchain networks, regulatory compliance frameworks, and AI-powered credit scoring documented in the Future Scope section.

FinWave is not merely a hackathon project. It is a proof of concept for a platform that, if scaled and regulated, could meaningfully reduce the cost of remittances, build financial resilience among underserved populations, and ultimately return more money to the families and communities that depend on migrant workers — the backbone of the global economy.

Team 20 believes that technology, when designed with empathy and engineered with excellence, has the power to create a more financially inclusive world.

21. Bibliography / References

#	Reference
[1]	IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, Institute of Electrical and Electronics Engineers, 1998.
[2]	World Bank Group, 'Remittance Prices Worldwide,' Quarterly Report, Q1 2024. [Online]. Available: https://remittanceprices.worldbank.org
[3]	Google Firebase, 'Firebase Authentication Documentation.' [Online]. Available: https://firebase.google.com/docs/auth
[4]	OpenAI, 'OpenAI API Reference, GPT-4o-mini.' [Online]. Available: https://platform.openai.com/docs
[5]	Google, 'Gemini API Documentation.' [Online]. Available: https://ai.google.dev/docs
[6]	MongoDB, 'MongoDB Atlas Documentation.' [Online]. Available: https://www.mongodb.com/docs/atlas
[7]	AWS Inc., 'Vercel Deployment Documentation.' [Online]. Available: https://docs.aws.amazon.com/
[8]	Render Inc., 'Render Web Services Documentation.' [Online]. Available: https://render.com/docs
[9]	Meta Open Source, 'React Documentation.' [Online]. Available: https://react.dev
[10]	Tailwind Labs, 'Tailwind CSS Documentation.' [Online]. Available: https://tailwindcss.com/docs
[11]	OpenJS Foundation, 'Express.js Documentation.' [Online]. Available: https://expressjs.com
[12]	Mongoose, 'Mongoose ODM Documentation.' [Online]. Available: https://mongoosejs.com/docs
[13]	i18next, 'react-i18next Documentation.' [Online]. Available: https://react.i18next.com
[14]	OWASP Foundation, 'OWASP Top Ten 2021.' [Online]. Available: https://owasp.org/Top10
[15]	W3C, 'Web Content Accessibility Guidelines (WCAG) 2.1,' W3C Recommendation, 2018. [Online]. Available: https://www.w3.org/TR/WCAG21
[16]	International Organization for Standardization, 'ISO/IEC 25010:2011 — Systems and software Quality Requirements and Evaluation (SQuaRE),' 2011.
[17]	NIST, 'NIST Special Publication 800-63B: Digital Identity Guidelines,' 2017. [Online]. Available: https://pages.nist.gov/800-63-3

— End of Document —

FinWave · Team 20 · SRS v1.0.0 · May 2026